



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 7

APIs: REST & GraphQL

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- What is an API?
- REST APIs – Core Concepts
- Django REST Framework (DRF)
- CRUD with Django REST Framework
- RESTful Best Practices
- Introduction to GraphQL
- Summary

Introduction: What is an API?

API stands for **Application Programming Interface**.

It defines how two systems communicate – just like how a waiter connects a customer and a kitchen.

Imagine you are in a restaurant:

- ✓ You (the frontend or mobile app) make a request (“I want pizza”).
- ✓ The waiter (API) delivers it to the kitchen (backend/database).
- ✓ The kitchen prepares the dish (data).
- ✓ The waiter brings it back in a structured way (JSON response).

Why APIs Are Important?

- ✓ They allow different applications to talk to each other.
- ✓ Used in mobile apps, web services, IoT, and AI systems.
- ✓ Enable integration between systems (e.g., PayPal, Google Maps, Twitter).

Introduction: What is an API?

Why We Need APIs in Web Development

Purpose	Description	Example
Data Exchange	Connect frontend (browser, mobile app) with backend (database).	The /students/ API sends student data to your React or Django frontend.
Integration	Allow systems from different vendors to work together.	Payment via PayPal API or login via Google OAuth.
Automation	Let scripts or services perform tasks automatically.	Auto-fetch weather or financial data daily.
Security	APIs control what data and actions are exposed – nothing else.	Users can update their profile but not system settings.
Scalability	APIs decouple client and server – each can evolve independently.	You can redesign the frontend without changing the backend.

Introduction: What is an API?

Without APIs...

Imagine a frontend directly accessing the database –

- ✓ Anyone could delete or change data accidentally.
- ✓ There would be no data validation, no permissions, no consistency.
- ✓ Changing database structure would break everything.

So instead, **APIs act as controlled gateways** – enforcing logic, validation, and access rules.

Introduction: Types of APIs?

Type	Description	Example
REST API	Resource-based, uses HTTP methods (GET, POST, PUT, DELETE)	/students/
GraphQL	Query-based, client defines what data it wants	{ student { name, grade } }
SOAP	XML-based, older style	Rarely used now

Question:

Why do you think JSON became the standard format for APIs instead of XML?

REST APIs – Core Concepts

REST (Representational State Transfer)

REST is an architectural style for building APIs that treat **data as resources**. Each resource (e.g., student, course, article) can be accessed via a **URL**.

REST Principles

Principle	Description
Stateless	Each request is independent; server doesn't remember previous requests.
Client-Server	Frontend (client) and backend (server) are separate.
Cacheable	Responses can be stored (cached).
Uniform Interface	Uses standard HTTP methods and predictable URLs.

REST APIs – Core Concepts

HTTP Methods

Method	Purpose	Example
GET	Read data	/students/ or /students/5/
POST	Create new data	/students/
PUT	Update existing data	/students/5/
DELETE	Remove data	/students/5/

When you visit:

http://127.0.0.1:8000/students/

you get:

Example (Without Django REST Framework)

views.py

```
from django.http import JsonResponse
from .models import Student

def student_list(request):
    students = list(Student.objects.values())
    return JsonResponse(students, safe=False)
```

```
[
  {"id": 1, "name": "Alice",
   "email": "alice@uwindsor.ca"},
  {"id": 2, "name": "Bob",
   "email": "bob@uwindsor.ca"}
]
```

This is a basic REST API response!

Django REST Framework (DRF)

Why Use Django REST Framework?

Django REST Framework (DRF) makes building APIs:

- ✓ Easier
- ✓ More secure
- ✓ More consistent
- ✓ Automatically browsable (for testing)

Installation:

```
pip install djangorestframework
```

Then add it to **settings.py**:

```
INSTALLED_APPS = [  
    ...,  
    'rest_framework',  
    'classroom',    # or your app  
]
```

Django REST Framework (DRF)

Create a Serializer

A serializer converts Django models ↔ JSON.

serializers.py

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'email']
```

Django REST Framework (DRF)

Create a View using DRF

views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Student
from .serializers import StudentSerializer

@api_view(['GET'])
def student_list(request):
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response(serializer.data)
```

Django REST Framework (DRF)

Add URL Route

urls.py

```
from django.urls import path
from . import views

urlpatterns += [
    path('api/students/', views.student_list,
name='student-list'),
]
```

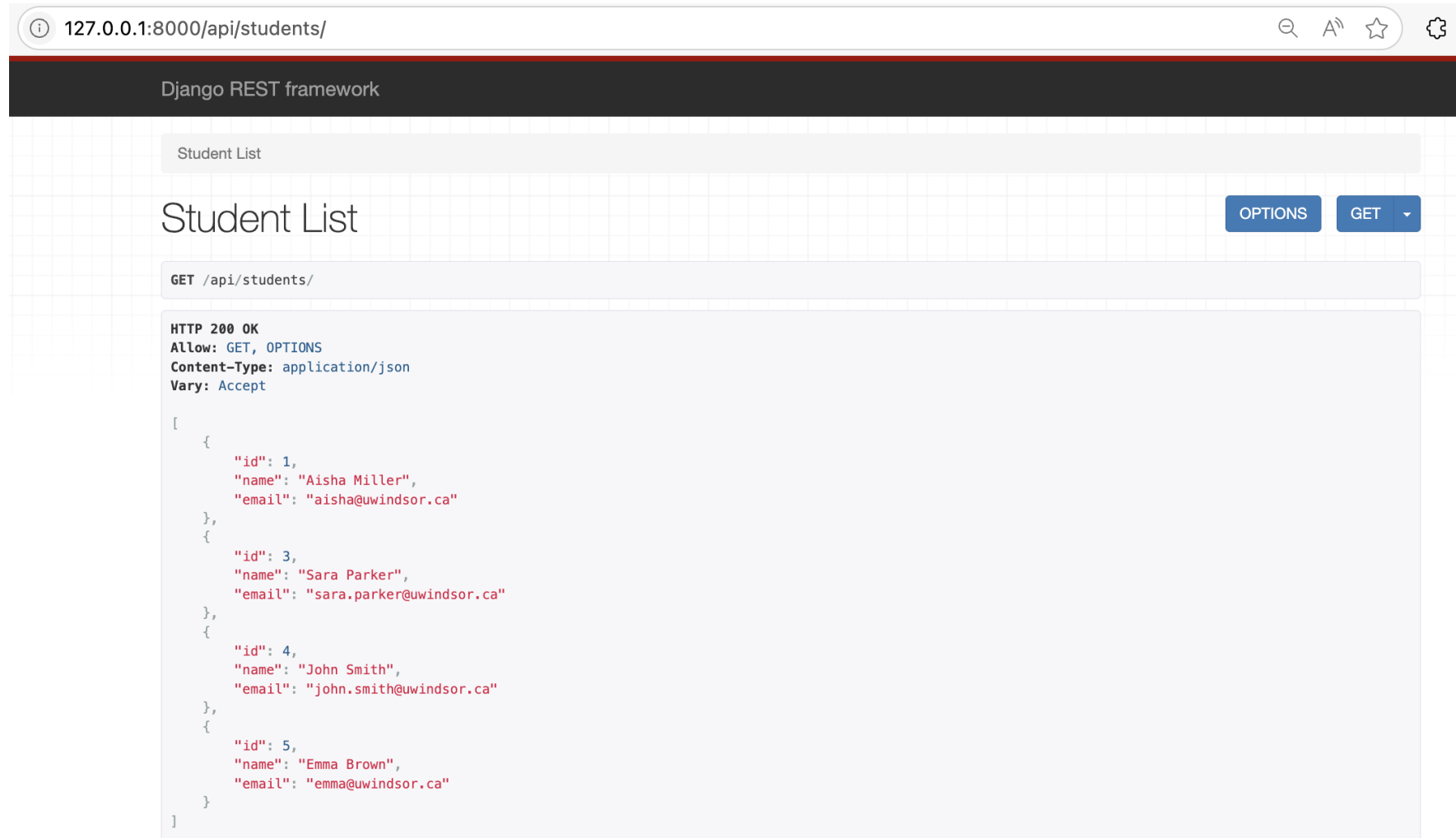
Test in Browser

python manage.py runserver

Visit: <http://127.0.0.1:8000/api/students/>

Django REST Framework (DRF)

**Test in
Browser
and see a
formatted
JSON
output:**



The screenshot shows a web browser at the URL `127.0.0.1:8000/api/students/`. The page title is "Django REST framework". The main content area is titled "Student List" and includes a "GET /api/students/" endpoint. The response is a JSON array of student objects, formatted for readability. The response status is "HTTP 200 OK" and the content type is "application/json".

```
HTTP 200 OK
Allow: GET, OPTIONS
Content-Type: application/json
Vary: Accept

[
  {
    "id": 1,
    "name": "Aisha Miller",
    "email": "aisha@uwindsor.ca"
  },
  {
    "id": 3,
    "name": "Sara Parker",
    "email": "sara.parker@uwindsor.ca"
  },
  {
    "id": 4,
    "name": "John Smith",
    "email": "john.smith@uwindsor.ca"
  },
  {
    "id": 5,
    "name": "Emma Brown",
    "email": "emma@uwindsor.ca"
  }
]
```

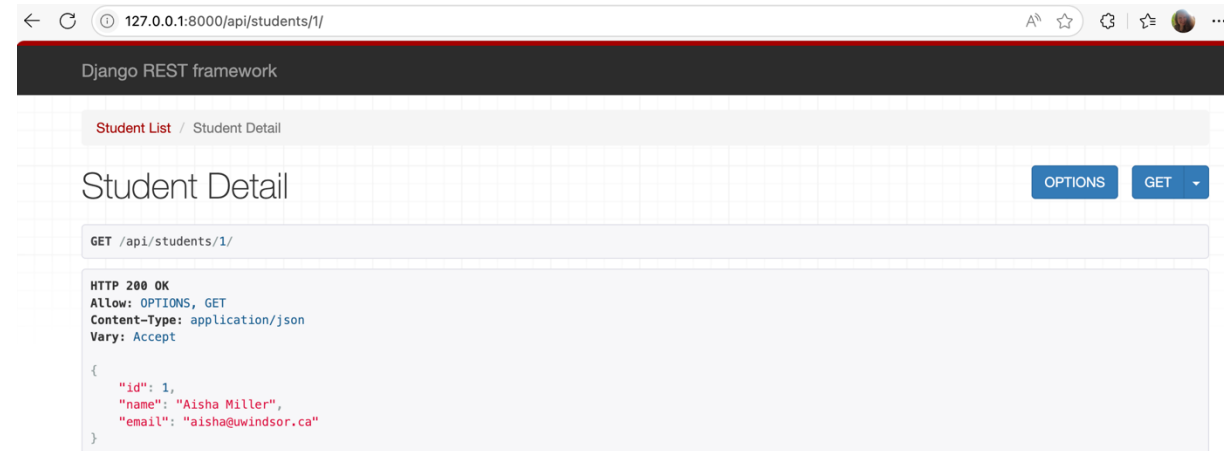

CRUD with Django REST Framework

GET (Retrieve a Single Record)

```
@api_view(['GET'])
def student_detail(request, pk):
    student = Student.objects.get(pk=pk)
    serializer = StudentSerializer(student)
    return Response(serializer.data)
```

URL:

```
path('api/students/<int:pk>/',
     views.student_detail),
```



CRUD with Django REST Framework

POST (Create)

```
@api_view(['POST'])
def student_create(request):
    serializer = StudentSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
    return Response(serializer.errors)
```

Put the view in your urls.py

```
path('api/students/create/',
    views.student_create, name='student_create'),
```

CRUD with Django REST Framework

Open the endpoint URL

`http://127.0.0.1:8000/api/students/create/`

127.0.0.1:8000/api/students/create/

Django REST framework

Student List / Student Create

Student Create

OPTIONS

GET /api/students/create/

HTTP 405 Method Not Allowed
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

```
{  
  "detail": "Method \"GET\" not allowed."  
}
```

Media type: application/json

Content:

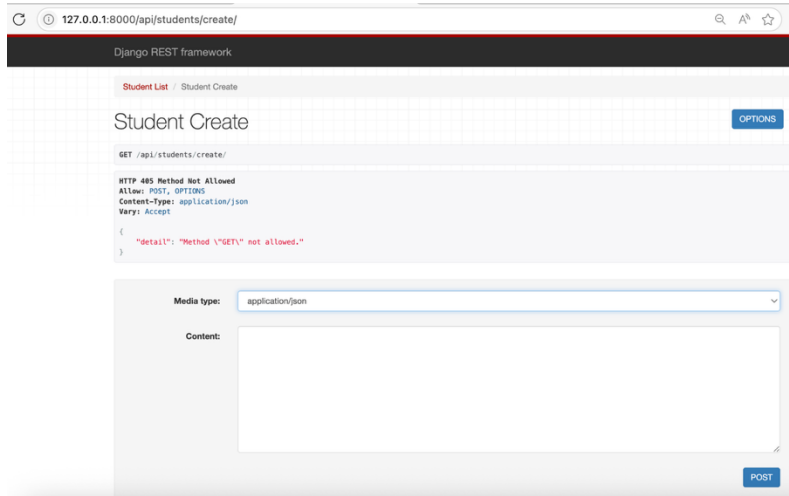
POST

That screen means your endpoint is correctly set up for **POST only**, but you opened it with **GET (in the browser)** – that’s why it says: “Method GET not allowed”

Now you just need to **send a POST request with JSON data**.

CRUD with Django REST Framework

Testing POST



In the box under “Content”, type valid JSON corresponding to the fields in your Student model.

```
{  
  "name": "Alice Johnson",  
  "email": "alice@example.com"  
}
```

If your serializer and view are set up properly, after you click **POST**, you should get a response like:

POST /api/students/create/

HTTP 200 OK
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

```
{  
  "id": 6,  
  "name": "Alice Johnson",  
  "email": "alice@example.com"  
}
```

CRUD with Django REST Framework

PUT (Update)

```
@api_view(['PUT'])
def student_update(request, pk):
    student = Student.objects.get(pk=pk)
    serializer = StudentSerializer(student, data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data)
    return Response(serializer.errors)
```

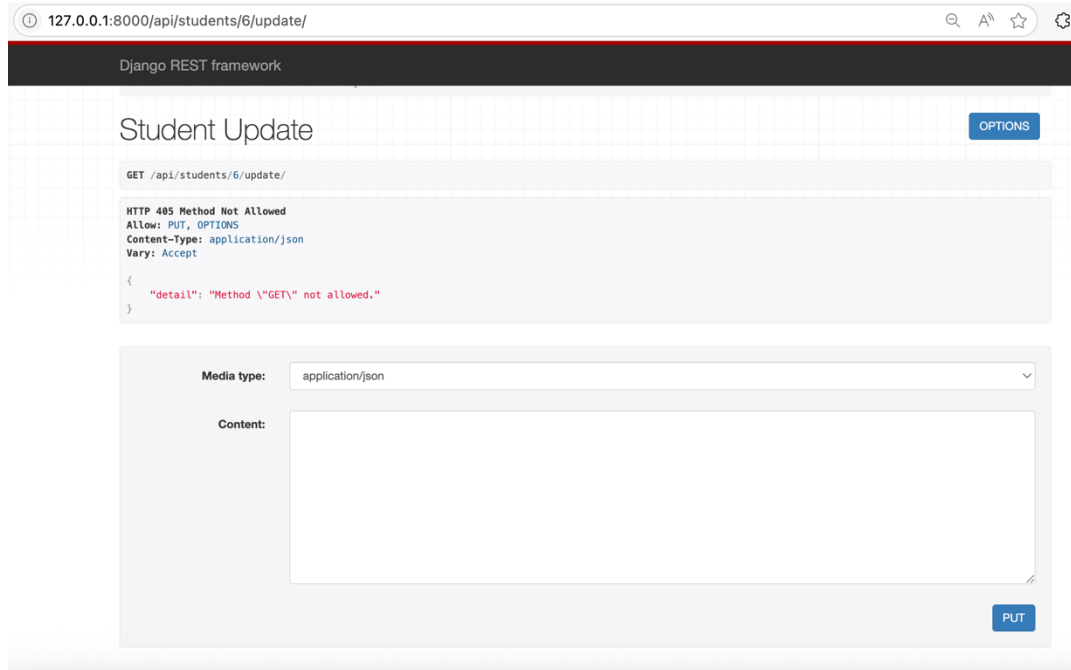
Put the view in your urls.py

```
path('api/students/<int:pk>/update/', views.student_update,
name='student_update'),
```

CRUD with Django REST Framework

Open the endpoint URL

`http://127.0.0.1:8000/api/students/6/update/`



Django REST framework

Student Update

OPTIONS

GET /api/students/6/update/

HTTP 405 Method Not Allowed
Allow: PUT, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "detail": "Method \"GET\" not allowed."
}

Media type: application/json

Content:

PUT

Type:

```
{  
  "name": "Alice Updated",  
  "email": "alice.new@example.com"  
}
```

You should receive:



Student Update

PUT /api/students/6/update/

HTTP 200 OK
Allow: PUT, OPTIONS
Content-Type: application/json
Vary: Accept

{
 "id": 6,
 "name": "Alice Updated",
 "email": "alice.new@example.com"
}

CRUD with Django REST Framework

DELETE

```
@api_view(['DELETE'])
def student_delete(request, pk):
    student = Student.objects.get(pk=pk)
    student.delete()
    return Response({'message': 'Student deleted successfully'})
```

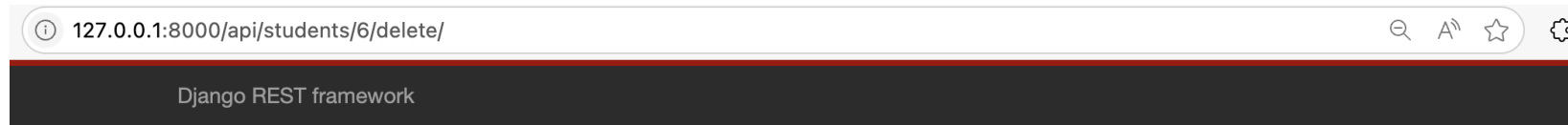
Put the view in your urls.py

```
path('api/students/<int:pk>/delete/', views.student_delete,
name='student_delete'),
```

CRUD with Django REST Framework

Open the endpoint URL

`http://127.0.0.1:8000/api/students/6/delete/`



[Student List](#) / [Student Detail](#) / [Student Delete](#)

Student Delete

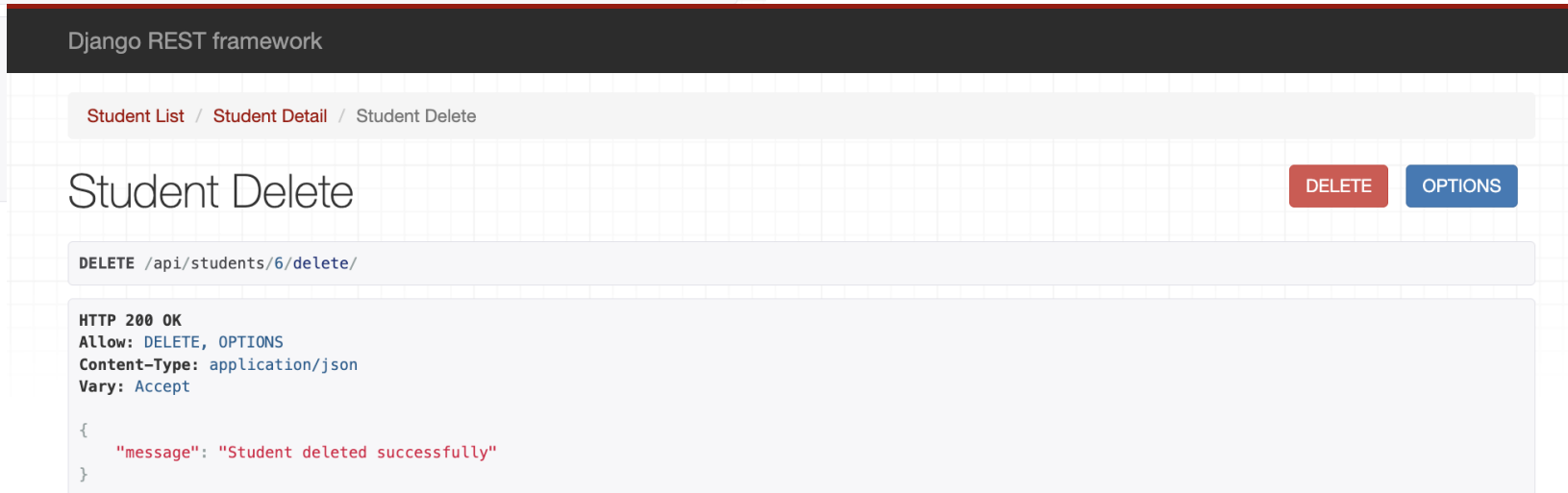
DELETE

OPTIONS

GET /api/students/6/delete/

HTTP 405 Method Not Allowed
Allow: DELETE, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "detail": "Method \"GET\" not allowed."
}
```



[Student List](#) / [Student Detail](#) / [Student Delete](#)

Student Delete

DELETE

OPTIONS

DELETE /api/students/6/delete/

HTTP 200 OK
Allow: DELETE, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "message": "Student deleted successfully"
}
```


RESTful Best Practices

Good API Design

Aspect	Example
Plural nouns	/students/ not /studentlist/
Consistent structure	/students/5/grades/
Meaningful status codes	200 OK, 201 Created, 404 Not Found
Versioning	/api/v1/students/
Documentation	Use DRF's built-in browsable API or Swagger

Follow these RESTful design principles to keep your API clear, predictable, and easy to maintain.

Consistent naming, structured URLs, proper status codes, and good documentation make APIs easier to use and extend.

RESTful Best Practices

Security and Permissions

In settings.py:

```
REST_FRAMEWORK = {  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ]  
}
```

What it does

- It **requires login (authentication)** for *every* API view by default.
- If a user is not logged in, any attempt to access your API will return:

```
{  
    "detail": "Authentication credentials were not provided."  
}
```

RESTful Best Practices

Security and Permissions

Case 1: Allow everyone (no login required)

If you want *some* endpoints to be public (e.g., GET or POST for testing), override the default:

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response

@api_view(['GET'])
@permission_classes([AllowAny])
def public_student_list(request):
    ...
```

This tells DRF: “Ignore global permissions for this view.”

RESTful Best Practices

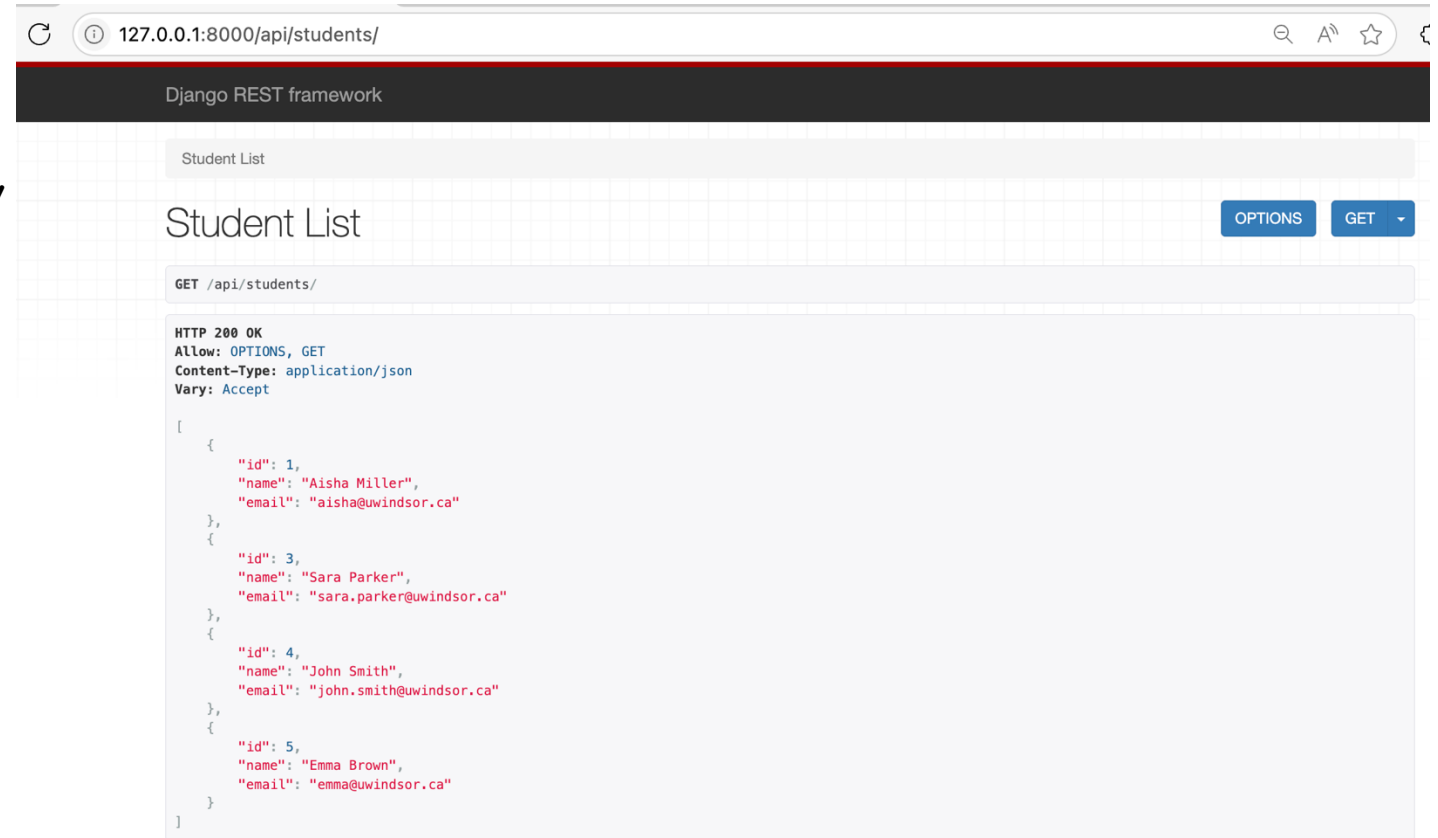
Security and Permissions

Case 1: Allow everyone (no login required)

Now when you visit:

`http://127.0.0.1:8000/api/students/`

...it will work for **anyone**, no login needed.



RESTful Best Practices

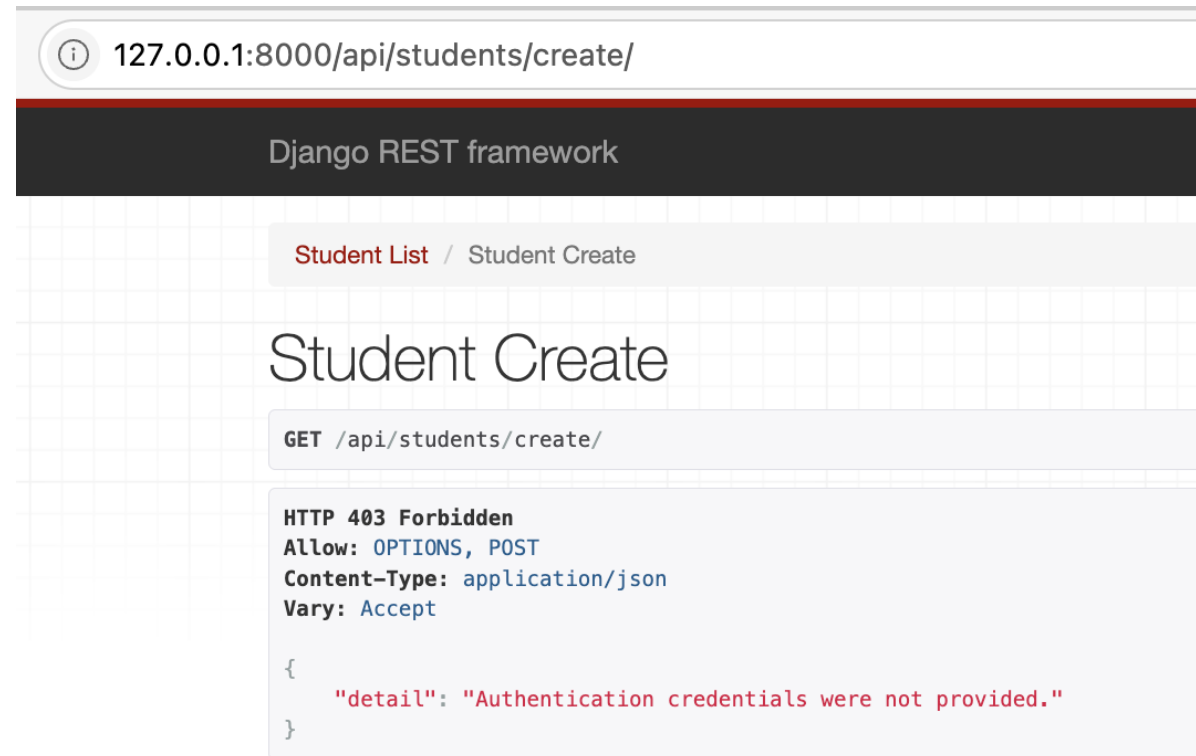
Security and Permissions

Case 2: Require login (use token or session)

To enforce authentication, keep your global setting as is.

Open the endpoint URL

```
http://127.0.0.1:8000/  
api/students/create/
```



RESTful Best Practices

Security and Permissions

Case 2: Require login (use token or session)

Go to:

`http://127.0.0.1:8000/admin/`

Enter your credentials:

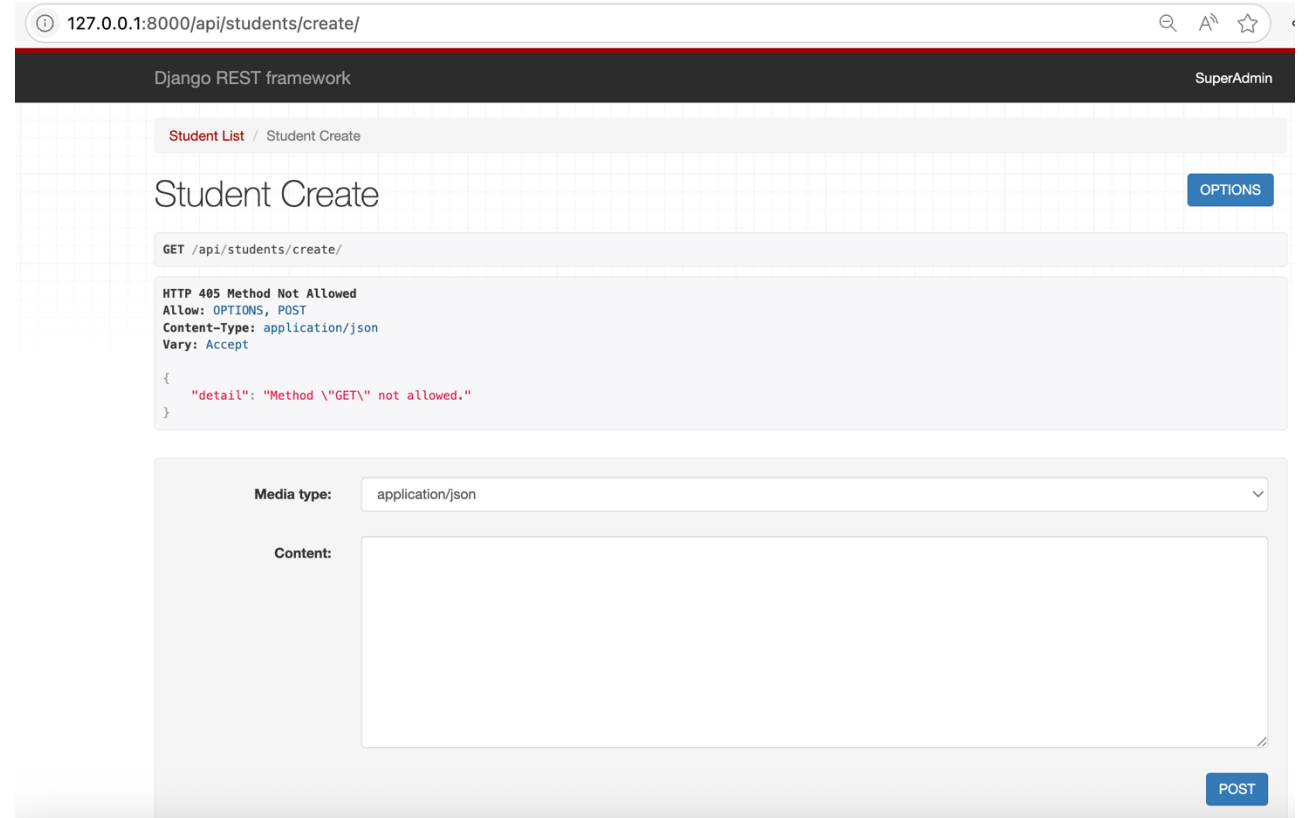
Username: **SuperAdmin**

Email: **SuperAdmin@uwindSOR.ca**

Password: **SuperPass123**

Then open the endpoint URL

`http://127.0.0.1:8000/
api/students/create/`



127.0.0.1:8000/api/students/create/

Django REST framework SuperAdmin

Student List / Student Create

Student Create

OPTIONS

GET /api/students/create/

HTTP 405 Method Not Allowed
Allow: OPTIONS, POST
Content-Type: application/json
Vary: Accept

{
 "detail": "Method \"GET\" not allowed."
}

Media type: application/json

Content:

POST

RESTful Best Practices

Group Discussion

Why should some API endpoints be public (e.g., blog posts), and others require authentication (e.g., user profiles)?

Introduction to GraphQL

What is GraphQL?

- ✓ A modern alternative to REST created by Facebook.
- ✓ Instead of multiple endpoints, GraphQL has **one endpoint** where clients specify **exactly what data they need**.

Example:

REST request:

```
GET /students/5/
```

Response:

```
{
  "id": 5,
  "name": "Alice",
  "email": "alice@uwindsor.ca",
  "courses": [...]
}
```

GraphQL query:

```
{
  student(id: 5) {
    name
    courses {
      title
    }
  }
}
```

Response:

```
{
  "student": {
    "name": "Alice",
    "courses": [
      {"title": "Math"},
      {"title": "C++"}
    ]
  }
}
```

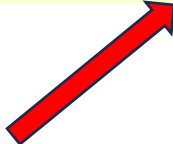

Introduction to GraphQL

Installing Graphene-Django (for GraphQL)

```
pip install graphene-django
```

settings.py

```
INSTALLED_APPS += ['graphene_django']  
GRAPHENE = {'SCHEMA': 'classroom.schema.schema' }
```



It tells Django **where to find your GraphQL schema**.

Graphene-Django is a library that lets you expose your Django models through **GraphQL APIs** (an alternative to REST APIs).

Instead of having multiple REST endpoints (/api/students/, /api/students/1/update/), you can use **one GraphQL endpoint** (e.g. /graphql/) and send **queries** or **mutations**

Introduction to GraphQL

Basic GraphQL Example schema.py

```
import graphene
from graphene_django.types import DjangoObjectType
from .models import Student

class StudentType(DjangoObjectType):
    class Meta:
        model = Student

class Query(graphene.ObjectType):
    all_students = graphene.List(StudentType)

    def resolve_all_students(self, info, **kwargs):
        return Student.objects.all()

schema = graphene.Schema(query=Query)
```

Introduction to GraphQL

Basic GraphQL Example

urls.py

```
from django.urls import path
from graphene_django.views import GraphQLView

urlpatterns += [
    path("graphql/", GraphQLView.as_view(graphiql=True)),
]
```

Do not forget to run the server!

python manage.py runserver

Now open:

<http://127.0.0.1:8000/graphql/>



Introduction to GraphQL

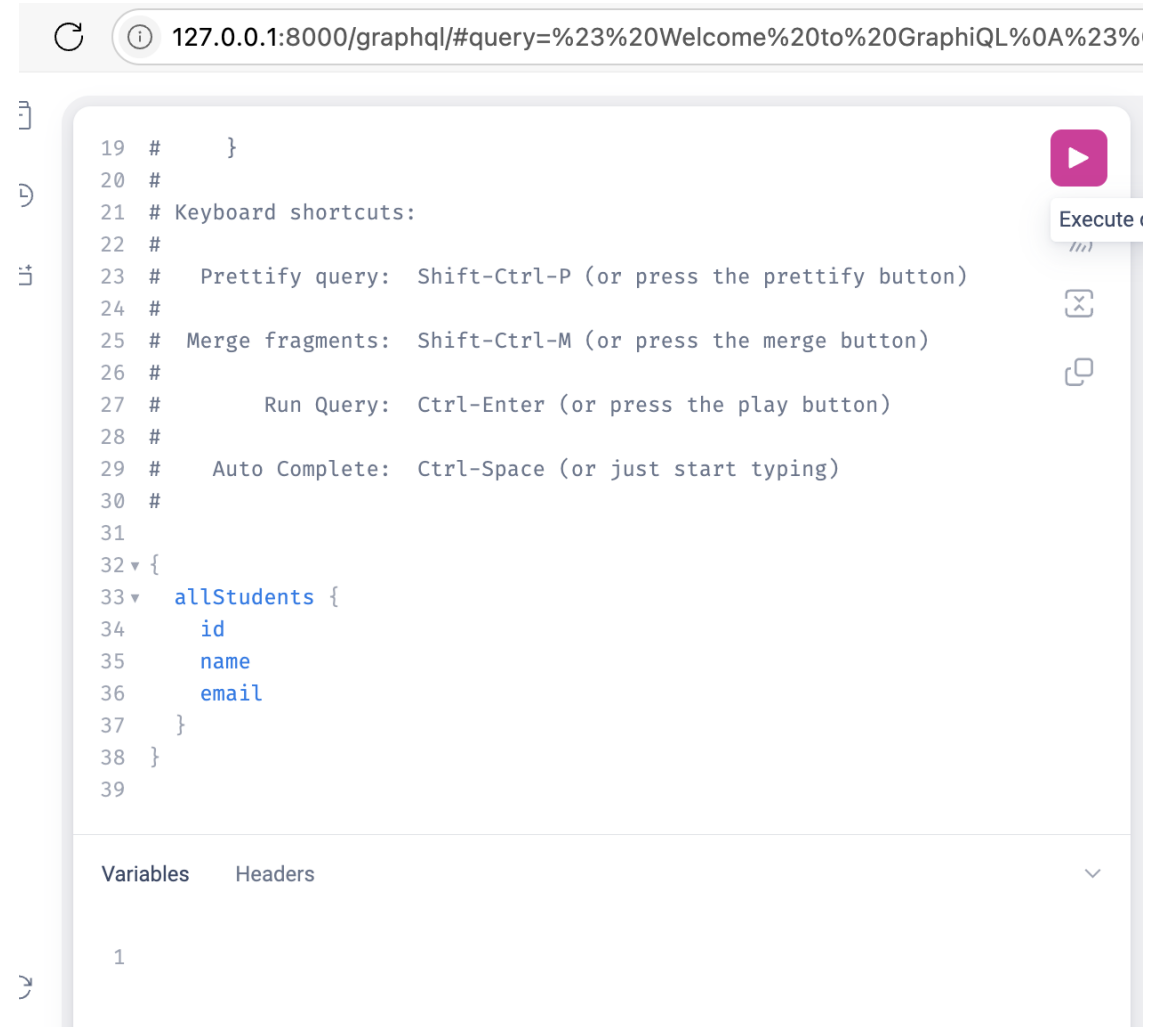
Basic GraphQL Example

Open:

`http://127.0.0.1:8000/graphql/`

Try:

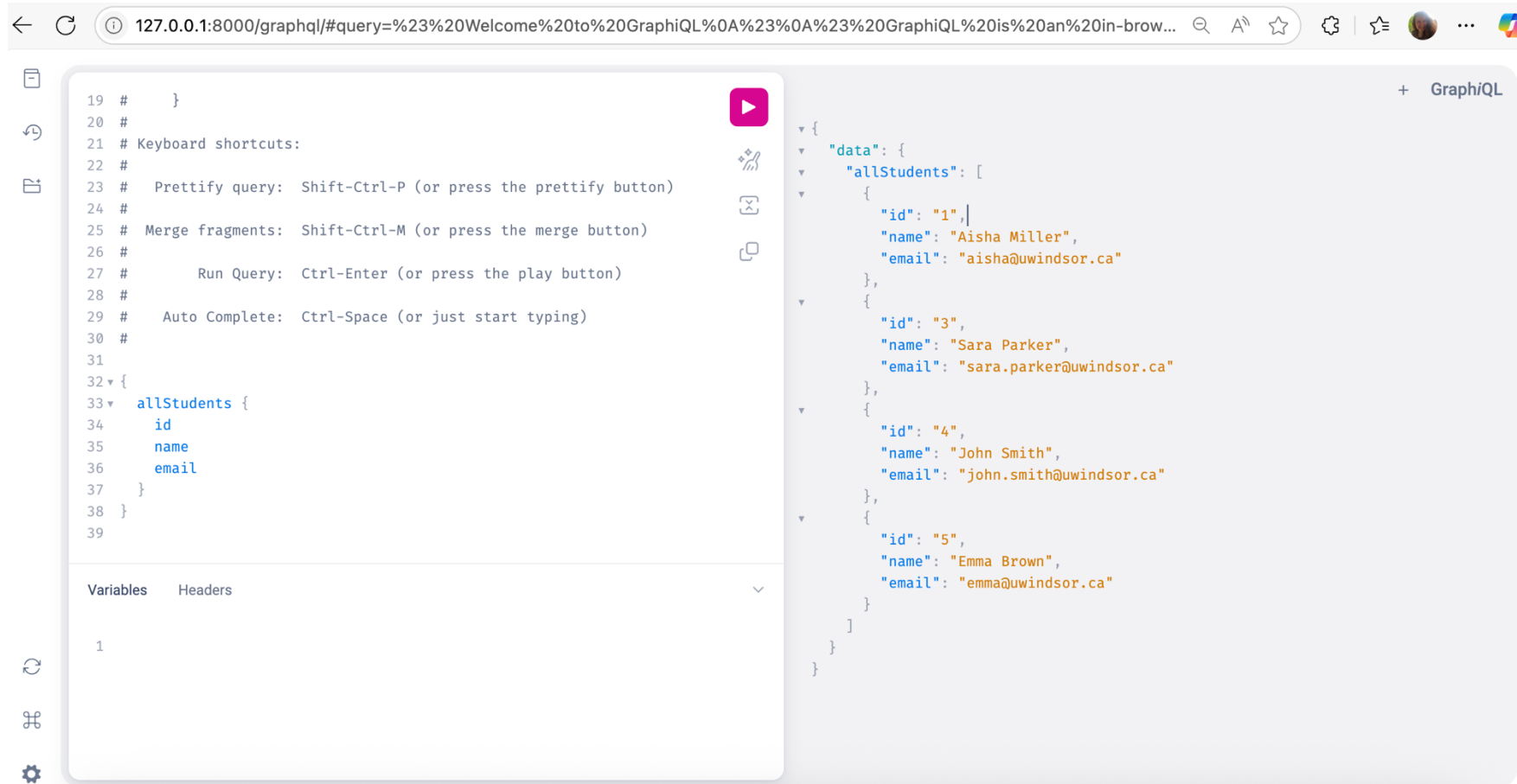
```
{
  allStudents {
    id
    name
    email
  }
}
```



Introduction to GraphQL

Basic GraphQL Example

Response:



The screenshot shows a web browser window with a GraphQL IDE. The URL bar shows a local address: 127.0.0.1:8000/graphql/#query=%23%20Welcome%20to%20GraphiQL%0A%23%0A%23%20GraphiQL%20is%20an%20in-brow... The IDE interface is split into two main panels. The left panel contains a query editor with line numbers 19 to 39. The query is a GraphQL query to fetch all students, with fields id, name, and email. The right panel shows the JSON response, which is a list of five student objects. The response is expanded to show the details of each student. The IDE also includes a sidebar on the left with icons for file explorer, search, and settings, and a bottom bar with a play button and other controls.

```
19 #  
20 #  
21 # Keyboard shortcuts:  
22 #  
23 #   Prettify query:  Shift-Ctrl-P (or press the prettify button)  
24 #  
25 #   Merge fragments: Shift-Ctrl-M (or press the merge button)  
26 #  
27 #   Run Query:      Ctrl-Enter (or press the play button)  
28 #  
29 #   Auto Complete:  Ctrl-Space (or just start typing)  
30 #  
31  
32 {  
33   allStudents {  
34     id  
35     name  
36     email  
37   }  
38 }  
39
```

```
{  
  "data": {  
    "allStudents": [  
      {  
        "id": "1",  
        "name": "Aisha Miller",  
        "email": "aisha@uwindsor.ca"  
      },  
      {  
        "id": "3",  
        "name": "Sara Parker",  
        "email": "sara.parker@uwindsor.ca"  
      },  
      {  
        "id": "4",  
        "name": "John Smith",  
        "email": "john.smith@uwindsor.ca"  
      },  
      {  
        "id": "5",  
        "name": "Emma Brown",  
        "email": "emma@uwindsor.ca"  
      }  
    ]  
  }  
}
```

Introduction to GraphQL

Using GraphQL

Define types and queries in classroom/schema.py:

```
import graphene
from graphene_django.types import DjangoObjectType
from .models import Student

# Define what our data looks like in GraphQL
class StudentType(DjangoObjectType):
    class Meta:
        model = Student
        fields = ("id", "name", "email")

# Define available queries
class Query(graphene.ObjectType):
    all_students = graphene.List(StudentType)
    student_by_id = graphene.Field(StudentType,
    id=graphene.Int(required=True))
```

Introduction to GraphQL

Using GraphQL

Define types and queries in `classroom/schema.py`:

```
# Resolver for all students
def resolve_all_students(root, info):
    return Student.objects.all()

# Resolver for one student
def resolve_student_by_id(root, info, id):
    return Student.objects.get(pk=id)

# Create schema
schema = graphene.Schema(query=Query)
```

Introduction to GraphQL

Using GraphQL

We have already connected the GraphQL View

```
from django.contrib import admin
from django.urls import path
from graphene_django.views import GraphQLView

urlpatterns = [
    path("admin/", admin.site.urls),
    path("graphql/", GraphQLView.as_view(graphiql=True)),
]
```

graphiql=True enables the interactive GraphQL editor in your browser.

Introduction to GraphQL

Using GraphQL

Then open: <http://127.0.0.1:8000/graphql/>

Query Examples (Equivalent to REST GET)

Get all students:

```
{
  allStudents {
    id
    name
    email
  }
}
```

Response:

```
{
  "data": {
    "allStudents": [
      {
        "id": "1",
        "name": "Aisha Miller",
        "email": "aisha@uwindsor.ca"
      },
      {
        "id": "3",
        "name": "Sara Parker",
        "email": "sara.parker@uwindsor.ca"
      },
      {
        "id": "4",
        "name": "John Smith",
        "email": "john.smith@uwindsor.ca"
      },
      {
        "id": "5",
        "name": "Emma Brown",
        "email": "emma@uwindsor.ca"
      }
    ]
  }
}
```

Introduction to GraphQL

Using GraphQL

Then open: <http://127.0.0.1:8000/graphql/>

Query Examples (Equivalent to REST GET)

Get one student by ID:

Response:

```
{  
  studentById(id: 1) {  
    id  
    name  
    email  
  }  
}
```

```
▼ {  
  ▼ "data": {  
    ▼ "studentById": {  
      "id": "1",  
      "name": "Aisha Miller",  
      "email": "aisha@uwindsor.ca"  
    }  
  }  
}
```

Introduction to GraphQL

Using GraphQL

Add Mutations (Equivalent to REST POST, PUT, DELETE)

Mutations in GraphQL are actions that change data.

Add mutation classes to schema.py

Update your **classroom/schema.py**:

....

```
# MUTATIONS
```

```
class CreateStudent(graphene.Mutation):  
    class Arguments:  
        name = graphene.String(required=True)  
        email = graphene.String(required=True)  
  
    student = graphene.Field(StudentType)  
  
    def mutate(root, info, name, email):  
        student = Student(name=name, email=email)  
        student.save()  
        return CreateStudent(student=student)
```

Introduction to GraphQL

Update your `classroom/schema.py`:

....

```
class UpdateStudent(graphene.Mutation):  
    class Arguments:  
        id = graphene.Int(required=True)  
        name = graphene.String()  
        email = graphene.String()  
  
    student = graphene.Field(StudentType)  
  
    def mutate(root, info, id, name=None, email=None):  
        student = Student.objects.get(pk=id)  
        if name:  
            student.name = name  
        if email:  
            student.email = email  
        student.save()  
        return UpdateStudent(student=student)
```

Introduction to GraphQL

Update your `classroom/schema.py`:

....

```
class DeleteStudent(graphene.Mutation):  
  
    class Arguments:  
  
        id = graphene.Int(required=True)  
  
        ok = graphene.Boolean()  
  
    def mutate(root, info, id):  
  
        student = Student.objects.get(pk=id)  
  
        student.delete()  
  
        return DeleteStudent(ok=True)
```

Introduction to GraphQL

Update your `classroom/schema.py`:

....

```
# Combine queries and mutations
class Query(graphene.ObjectType):
    all_students = graphene.List(StudentType)
    student_by_id = graphene.Field(StudentType, id=graphene.Int(required=True))

    def resolve_all_students(root, info):
        return Student.objects.all()

    def resolve_student_by_id(root, info, id):
        return Student.objects.get(pk=id)

class Mutation(graphene.ObjectType):
    create_student = CreateStudent.Field()
    update_student = UpdateStudent.Field()
    delete_student = DeleteStudent.Field()

schema = graphene.Schema(query=Query, mutation=Mutation)
```

Introduction to GraphQL

Run Equivalent Queries and Mutations:

Now open: <http://127.0.0.1:8000/graphql/>

+ Create Student (POST)

```
mutation {  
  createStudent(name: "Carla", email: "carla@uwindsor.ca") {  
    student {  
      id  
      name  
      email  
    }  
  }  
}
```

Introduction to GraphQL

Run Equivalent Queries and Mutations:

Now open: <http://127.0.0.1:8000/graphql/>

✗ Delete Student (DELETE)

```
mutation {  
  deleteStudent(id: 7) {  
    ok  
  }  
}
```

Discussion:

1. Why is it useful to have only **one endpoint** in GraphQL?
2. Which approach (REST or GraphQL) would be easier to use in a **mobile app** where you only need limited data per screen?
3. Does GraphQL replace REST?

Summary – REST vs GraphQL

Feature	REST	GraphQL
Endpoints	/api/students/ /api/students/<id>/	Single: /graphql/
Requests	GET, POST, PUT, DELETE	Query, Mutation
Format	Multiple endpoints	One endpoint, flexible
Response	Fixed	Custom (only what you ask for)
Over-fetching	Common	None
Under-fetching	Common	None
Testing tool	Postman, Browser	GraphiQL Playground